

BUGMXT SI

Bug Triage Report

5-layer code integrity scan · ATANDA Command Centre · UX polish + ontology cleanup (778e73d..169519b)

BUGMXT SI — Bug Triage Report

Target: ATANDA Command Centre — commits 778e73d..169519b (Engine button glow + HoverCard explainers + ontological-alignment cleanup)

Engine: BUGMXT SI v1.0 (JCSE 46 / Platinum)

Model: claude-sonnet-4-6 (4-phase scan)

Scan duration: 153.9s total

Tokens: 61720 in / 6845 out

Generated: 2026-05-23T21:08:32.079Z

Executive Summary

The two change sets are largely sound and the ontological corrections (cartridge fence wording, badge suffix removal, IPDD/PWDD explainer) are accurate and well-targeted. The primary risks missed by self-review are in the JCSE HoverCard: the 7-pillar enumeration in the explainer copy is wrong in both item count and naming against the authoritative Context Craft rubric, and the tier-band ranges in the hover text are inconsistent with the tier-band ranges hard-coded in the `tier` computation immediately above. One CSS scoping issue is present: the new keyframe and utility classes are declared inside a `@layer` or ruleset block whose closing brace placement must be verified, and both animations share an identical duration/easing but are defined as independent `@keyframes` with no shared custom property — a maintainability smell that will cause drift if timing is changed. No authorisation, PDD-orphan, or blast-radius issues were introduced; the `nextEngineId` filter correctly caps at `id <= 7`, excluding side-step engines.

Layer 1 — Syntax Sweep

- **[SYNTAX-001]** | Severity: LOW | File: `artifacts/command-centre/src/components/shared/JCSECounter.tsx` | Lines: 56–58

The `tier` ternary chain uses bare expression lines without wrapping parentheses and relies on implicit operator-precedence across four conditional branches. This is valid JavaScript/TypeScript but will fail `prettier` default formatting (it will reformat to a single line exceeding the project's likely `printWidth`), and several TSC strict-mode configs with `@typescript-eslint/no-nested-ternary` enabled will reject it.

Recommendation: Wrap in an explicit `if/else if` block or an IIFE, or annotate with `// eslint-disable-next-line @typescript-eslint/no-nested-ternary` if the rule is active.

- **[SYNTAX-002]** | Severity: MEDIUM | File: `artifacts/command-centre/src/index.css` | Lines (new block, ~L215–L240)

Both `@keyframes feature-pulse` and `@keyframes feature-ring` are declared inside the outermost `@layer base { ... }` (or equivalent Tailwind layer block) that wraps the rest of `index.css`. `@keyframes` rules are **not** valid inside a `@layer` block in CSS Cascade Layers spec (they must be declared at the top level or inside a named `@layer` that explicitly permits rules). If the surrounding block is a Tailwind `@layer base` or `@layer utilities` directive, placing `@keyframes` inside it is non-standard and browser-inconsistent —

Safari "d16 and Firefox "d117 will silently ignore keyframes declared inside `@layer` base. **Recommendation:** Hoist both `@keyframes` declarations to the top level (outside any `@layer` block); keep only the `.animate-feature-pulse` and `.animate-feature-ring` utility class rules inside the layer block, or move everything into `@layer utilities`.

- **[SYNTAX-003]** | Severity: LOW | File: `artifacts/command-centre/src/components/shared/FeatureNavItem.tsx` | Line: ~52 (new render block)

The `data-next` attribute is set to the string `"true"` when `showAttention` is truthy and undefined when falsy. Passing undefined as a JSX attribute value causes React to **omit** the attribute entirely — this is the intended behaviour, but TypeScript with `strictNullChecks` and the DOM `data-*` attribute typings will surface a `Type 'undefined' is not assignable to type 'string'` error if the project's `tsconfig` targets DOM lib with strict mode. **Recommendation:** Use `data-next={showAttention || undefined}` (already the pattern here) but annotate with an explicit cast or use `{...(showAttention ? { 'data-next': 'true' } : {})}` to satisfy strict DOM typings without suppression.

- **[SYNTAX-004]** | Severity: INFO | File: `artifacts/command-centre/src/pages/session-detail.tsx` | Line: ~81

The `nextEngineId` constant is declared with `const` inside the component body, between two other `const` declarations, but its value depends on `ENGINES` (module-level constant) and `getFeatureStatus` (a closure over `featureStates`). This is syntactically correct. However, `getFeatureStatus` is defined **below** `nextEngineId` in the diff hunk (the function body appears after line 77 and `nextEngineId` is at ~81 in the same render scope). Verify in the full file that `getFeatureStatus` is hoisted (i.e., defined as a `function` declaration, not a `const` arrow). If it is a `const` arrow, temporal dead zone rules will cause a runtime `ReferenceError`. The diff does not show the full ordering — flag for architect review.

- No import-level, delimiter, or operator syntax errors were found in the remaining changed files (`cartridge-context.ts`, `badges.ts`, `constants.ts`).

Layer 2 — Logic & Outcome Audit

[LOGIC-001] `nextEngineId` computed only once on render — stale after in-flight status mutation

The `nextEngineId` computation calls `getFeatureStatus(e.id)` for every engine in the F1–F7 slice at the moment the component renders, then stores the result in a plain `const`. `getFeatureStatus` reads from the TanStack Query cache (`state?.status`). When an engine action resolves and the query is invalidated, the cache is updated, but `nextEngineId` is **not recalculated as a derived memo**. Between the moment the cache is invalidated and the moment React schedules the re-render, there is a window where the NEXT badge still points to an engine that is now COMPLETE, or — if two query invalidations land in the same batch — skips the true next engine entirely.

- **File:** `artifacts/command-centre/src/pages/session-detail.tsx`
- **Function/area:** `nextEngineId` declaration (~line 81) and `renderSequenceList` callback
- **Expected:** `nextEngineId` always reflects the current query cache state; the NEXT badge is always on the lowest-id AVAILABLE engine at paint time.
 - **Actual:** Transient stale renders are possible; in async-concurrent React 18 (`startTransition` paths, Suspense boundaries) the stale `const` can survive across multiple paint cycles if the parent does not unmount.
 - **Fix pathway:** Patch — wrap the derivation in `useMemo` keyed on the `featuresState` query data reference:

```
const nextEngineId = useMemo(() =>
  ENGINES.filter((e) => e.id <= 7)
    .find((e) => getFeatureStatus(e.id) === FeatureStatus.AVAILABLE)?.id,
  [featuresState] // replace with actual query-data variable name
);
```

[LOGIC-002] `isNext` prop can be `true` on a COMPLETE engine when F1–F7 have all completed

`nextEngineId` uses `.find()` which returns `undefined` when every F1–F7 engine is COMPLETE. `engine.id === nextEngineId` then evaluates to `engine.id === undefined`, which is always false, so no button is decorated — that part is correct. However, `showAttention` inside `FeatureNavItem` is computed as `isNext && !isActive && !isLocked`. If the parent ever passes `isNext={true}` for a COMPLETE engine (e.g., the engine was AVAILABLE at call-site evaluation, the user clicked before the query resolved, and the prop hasn't updated yet), `showAttention` becomes true because `isLocked` is false and `isActive` may be false if the user has navigated away — meaning a COMPLETE engine shows the pulsing NEXT glow and the "!" Click to run this stage" prompt.

- **File:** `artifacts/command-centre/src/components/shared/FeatureNavItem.tsx` (~line 43)
- **Function:** `FeatureNavItem` — `showAttention` derivation
- **Expected:** COMPLETE engines never show the NEXT badge or attention animation.
- **Actual:** A COMPLETE engine can transiently show NEXT styling if `isNext` hasn't updated before status transitions.
- **Fix pathway:** Patch — add `status !== FeatureStatus.COMPLETE` to the `showAttention` guard:

```
const showAttention =
  isNext &&
  !isActive &&
  !isLocked &&
  status !== FeatureStatus.COMPLETE;
```

[LOGIC-003] `nextEngineId` filter boundary `e.id <= 7` silently excludes F8 (id: 9) but the id sequence is not contiguous — id 8 is F6-VDJ

The filter `ENGINES.filter((e) => e.id <= 7)` is intended to exclude side-step engines F6-VDJ (id 8) and F8/CODE DJ (id 9). This is correct by the comment's stated intent. However, the `ENGINES` array uses non-contiguous ids: F7 has `id: 8` (zero-indexed position 7 = id 8 in the array)... checking the diff — F7 is `id: 8`? Re-reading: `id: 1` through `id: 7` for F1–F7, then `id: 8` for F6-VDJ, then `id: 9` for F8. The filter `e.id <= 7` **correctly** includes only F1–F7. But this is fragile: if any future engine is inserted into the pipeline with an `id` that is actually a side-step, or if F7 is ever renumbered, the guard silently breaks. More critically, F8 is named "F8" but carries `id: 9` — the naming and the `id` are already misaligned, which means any code that maps display-name to `id` (e.g., test selectors using `data-testid="feature-nav-f8"`) will resolve to `id 9` while a reader expecting `F8 = id 8` will be confused and may introduce off-by-one errors in future status checks.

- **File:** `artifacts/command-centre/src/lib/constants.ts` (`ENGINES` array) and `artifacts/command-centre/src/pages/session-detail.tsx` (~line 81)
- **Expected:** Engine `id` and display name are unambiguously aligned; pipeline boundary is expressed as an explicit allowlist, not a numeric comparison against a non-contiguous sequence.
- **Actual:** The `id`-to-name mapping has a gap (no `id 8` in the linear sequence; `F8 = id 9`), and the pipeline boundary relies on the numeric accident that all linear engines currently happen to have `ids <= 7`.
- **Fix pathway:** Refactor — introduce an `isPipelineStage: boolean` field on each `ENGINES` entry (or a dedicated `PIPELINE_ENGINE_IDS` constant) and filter on that field rather than `e.id <= 7`. Also align F8's `id`

to 8 and F6-VDJ's id to a value that makes the non-pipeline status self-documenting (e.g., a string key or a dedicated namespace).

[LOGIC-004] HoverCard on `INGESTED !` PWDD` badge mis-states the PWDD acronym expansion

The HoverCard body reads "*PromptWare Design Document*" as the expansion of PWDD. Per the authoritative context provided (replit.md ontology), PWDD = **PWDD** is the HARNESS-certified OUTPUT — the correct expansion as used in the ecosystem is **PromptWare Design Document** for PWDD and **Ingestion Product Design Document** for IPDD. That part is consistent. However, the HoverCard header label reads "PWDD output session" without expanding PWDD in the header, while the body expands IPDD inline. This asymmetry means a user who reads only the header label still does not know what PWDD stands for. More substantively, the body copy says "*Its terminal F7 artefact is a HARNESS-certified PWDD, not a regular MVP-PDD*" — this is logically correct per the ontology but introduces a subtle implication that PWDD and MVP-PDD are mutually exclusive types, when

Layer 3 — HARP (Human + AI Readability)

- [HARP-AI-001] | Severity: MEDIUM | File: constants.ts | ENGINES array — F1 JCSE pillar list contradicts canonical rubric

The explainer string for F1 reads: "*Scores your raw idea on the 7-pillar JCSE rubric (Specificity, Role, Intent, Domain, Format, Examples, Constraints)*". The canonical 7 pillars per the JCSE HoverCard added in JCSECounter.tsx (same diff) are: **SYSTEM, ROLE, INSTRUCTION, EXAMPLE, CONSTRAINT, FORMAT, DATA**. The F1 explainer invents entirely different pillar names (Specificity, Intent, Domain) that appear nowhere else in the codebase and contradict the authoritative list now surfaced in the JCSE chip tooltip. An AI agent consuming constants.ts could infer a different rubric than one consuming JCSECounter.tsx. These two artefacts are now in contradiction within the same changeset.

Recommendation: Align the F1 explainer to the canonical 7: "...across the 7-pillar JCSE rubric: SYSTEM, ROLE, INSTRUCTION, EXAMPLE, CONSTRAINT, FORMAT, DATA..."

- [HARP-AI-002] | Severity: MEDIUM | File: constants.ts | ENGINES entry id=8 (F6-VDJ) — id/name coupling is ambiguous for agents

Engine id=8 carries name: "F6-VDJ" — the numeric id field suggests position 8 in the sequence, but the name prefix F6 implies it branches from stage 6. A future agent traversing the ENGINES array by id would treat this as stage 8 in a linear sequence; one traversing by name prefix would treat it as a sibling of F6. The nextEngineId filter in session-detail.tsx guards `e.id <= 7` to exclude this engine, but that guard is implicit tribal knowledge — it is not documented as a property of the engine itself. If a future agent adds a feature and forgets the `<= 7` convention, F6-VDJ and F8 will silently enter the NEXT-detection logic.

Recommendation: Add a boolean field `isSideStep: true` to the F6-VDJ and F8 ENGINES entries and drive the `nextEngineId` filter off that field rather than a magic id boundary. Add a JSDoc block above the ENGINES constant explaining the `isSideStep` semantic.

- [HARP-AI-003] | Severity: LOW | File: session-detail.tsx | `nextEngineId` comment says "F1!F7" but the boundary guard is `id <= 7`, not `id <= 8`

The inline comment reads "*Side-step engines (F6-VDJ, F8) are never marked NEXT*". F8 has `id: 9`, so `id <= 7` correctly excludes both F6-VDJ (`id=8`) and F8 (`id=9`). However, F8 is not named "F8" as an `id` value — it is `id: 9`. An agent reading the comment alone would assume `id=8` is the excluded ceiling for F8, which is wrong. The comment conflates the display name ("F8") with the array `id` (9). This will mislead any AI agent that tries to derive the exclusion rule from the comment rather than the code.

Recommendation: Rewrite the comment: "*Side-step engines are those with `isSideStep: true` (currently F6-*

VDJ id=8, F8 id=9). The ``id <= 7`` guard encodes this — if the `ENGINES` array changes, update this filter."

- **[HARP-AI-004] | Severity: LOW | File: FeatureNavItem.tsx | No JSDoc on the exported component; ``isNext`` and ``explainer`` props are undocumented**

`FeatureNavItem` is a cross-cutting shared component consumed in at least two render paths (`renderSequenceList` in `session-detail.tsx`; potentially the Sheet variant). Neither `isNext` nor `explainer` carry any inline documentation. An AI agent generating a new call-site for this component has no machine-readable signal that `isNext` is a computed/derived flag (it should never be hardcoded `true` from a static list) or that `explainer` is intentionally optional (falls back to description-only display). The component's "NEXT" UX behaviour — pulse animation, ring, badge, CTA copy — is high-impact and could be accidentally activated by a future agent passing `isNext={someBoolean}` without understanding the sequencing contract.

Recommendation: Add a JSDoc block to `FeatureNavItemProps` and to the exported function specifying: `isNext` is derived from the session feature-state pipeline and MUST NOT be statically set; `explainer` is engine-level copy from `ENGINES` constant, not user-supplied.

- **[HARP-HUMAN-001] | Severity: LOW | File: session-detail.tsx | Badge text changed from ``INGESTED`` to ``INGESTED !`` PWDD without a changelog note**

The visible badge label was `INGESTED · PWDD` (middle dot separator) and is now `INGESTED !` PWDD` (arrow). The arrow implies a transformation/flow which is semantically accurate, but the change is a user-visible string alteration that is not called out anywhere in the diff's comments or commit context. A human reviewing this diff cold may read the `!`` as a rendering artifact or an incomplete migration to a two-chip design. The diff comment only references the `HoverCard` upgrade, not the label change.

Recommendation: Add a brief inline comment: `{/* Arrow denotes input!`output transformation: IPDD ingested, PWDD produced */}` adjacent to the badge text, or note it explicitly in the PR description.

- **[HARP-HUMAN-002] | Severity: LOW | File: index.css | ``animate-feature-ring`` keyframe ends at ``scale(1.04)`` with ``opacity: 0`` and stays there — no return to baseline**

The `feature-ring` keyframe goes `0% !` 70% !` 100%`` both 70% and 100% share `opacity: 0; transform: scale(1.04)`. The animation will snap back to `opacity: 0.65; scale(1)` at the next iteration start (0%) — this creates a visible flash/jump at the loop boundary because there is no easing back to the initial state. The `feature-pulse` animation (which runs simultaneously on the same element) correctly returns to its 0%-state. A human developer reading these two keyframe blocks back-to-back would assume they are designed symmetrically, but they are not: `feature-pulse` is symmetric, `feature-ring` is a one-way expand-and-fade that jumps on loop.

Recommendation: This is likely intentional (ring expands and fades once per cycle, then resets), but add a comment: `/* one-shot expand-fade per cycle – intentional snap-reset at loop boundary */` to prevent a future developer from "fixing" it into a symmetric ease that breaks the visual effect.

- ****[HARP-AI-005] | Severity: LOW | File: JCSECounter.tsx | `INSTRUCTION` (max 8) called out in H**

Layer 5 — EAL Bayesian Triage

All issue IDs carried forward from Layers 1–4 as provided. `[LOGIC-004]` and `[HARP-AI-005]` were truncated mid-sentence in the prior findings; their scores are assessed on the basis of the stated severity and partial content — noted inline.

ID	Layer	Severity (1-5)					Likelihood (1-5)					Blast Radius (1-5)					Detectability (1-5)					EAL				
Score	Priority																									
--- --- --- --- --- --- ---																										
[HARP-AI-001]		3	4	5	4	4		**4.25**		**CRITICAL**																
[LOGIC-001]		2	4	4	3	4		**3.85**		**HIGH**																
[LOGIC-003]		2	3	5	4	5		**3.85**		**HIGH**																
[HARP-AI-002]		3	3	4	4	4		**3.55**		**HIGH**																
[SYNTAX-002]		1	4	3	3	3		**3.30**		**HIGH**																
[LOGIC-002]		2	3	4	2	4		**3.25**		**HIGH**																
[LOGIC-004]		2	3	5	2	4		**3.25**		**HIGH**																
[SYNTAX-001]		1	2	4	2	3		**2.60**		**MEDIUM**																
[HARP-AI-003]		3	2	3	3	4		**2.75**		**MEDIUM**																
[HARP-AI-004]		3	2	3	3	4		**2.75**		**MEDIUM**																
[HARP-AI-005]		3	2	3	2	4		**2.60**		**MEDIUM**																
[SYNTAX-004]		1	4	2	3	2		**2.55**		**MEDIUM**																
[HARP-HUMAN-002]		3	2	3	2	4		**2.55**		**MEDIUM**																
[SYNTAX-003]		1	2	3	2	3		**2.35**		**LOW**																
[HARP-HUMAN-001]		3	1	5	1	5		**2.30**		**LOW**																

EAL Score formula: (Severity × 0.35) + (Likelihood × 0.30) + (Blast Radius × 0.20) + (Detectability × 0.15). All inputs are 1–5 integer scales.

Bug Triage Board

1. [HARP-AI-001] — F1 explainer pillar list contradicts the canonical JCSE rubric surfaced in the same changeset

- **Severity:** HIGH (4/5)
- **Blast Radius:** HIGH — Any AI agent, operator, or user who reads the F1 explainer copy internalises a different 7-pillar rubric (Specificity, Role, Intent, Domain, Format, Examples, Constraints) than the one the JCSE HoverCard chip now authoritatively defines (SYSTEM, ROLE, INSTRUCTION, EXAMPLE, CONSTRAINT, FORMAT, DATA). The contradiction is baked into the same shipped changeset, so both the incorrect definition and the correct one become simultaneously visible in the same UI session.
- **Estimated Fix Time:** 15 minutes
- **Fix Pathway:** Patch — edit the `explainer` string on the F1 entry in `constants.ts` to read: *"...across the 7-pillar JCSE rubric: SYSTEM, ROLE, INSTRUCTION, EXAMPLE, CONSTRAINT, FORMAT, DATA..."*
- **Regression Risk:** None — string-only change to a display constant; no logic, query, or API surface affected.
- **Why this matters:** This changeset explicitly added the JCSE HoverCard to establish the authoritative pillar definitions in the UI. Shipping F1 explainer copy that invents different pillar names in the same release creates a trust-breaking ontological contradiction: two simultaneous UI elements on the same screen will disagree on what JCSE measures. It will mislead both users calibrating their prompts and any AI agent consuming `constants.ts` as a cold context source.

2. [LOGIC-001] — ``nextEngineId`` is a plain ``const``, not a ``useMemo``; stale between query invalidation and re-render

- **Severity:** HIGH (4/5)
- **Blast Radius:** MEDIUM — Affects every session-detail page render during the window between a status-mutation resolution (e.g., engine run completes, status transitions AVAILABLE !' IN_PROGRESS !' COMPLETE) and the scheduled React re-render. Under React 18 concurrent mode or `startTransition` wrapping, this window is non-trivial. The NEXT badge can persist on a now-COMPLETE engine or be absent from the true next engine, silently giving the user incorrect navigational guidance.
- **Estimated Fix Time:** 20 minutes

- **Fix Pathway:** Patch — replace the bare `const nextEngineId = ENGINES.filter(...).find(...)?.id` with `const nextEngineId = useMemo(() => ENGINES.filter(...).find(...)?.id, [/* featuresState query data ref */])`. Add `useMemo` to the React import line.
- **Regression Risk:** Low — `useMemo` with the correct dependency array produces identical values to the current `const` on every full re-render, and eliminates stale values between renders. The only regression surface is an incorrect dependency array; architect should verify the exact query-data variable name.
- **Why this matters:** The NEXT badge is the primary navigational affordance for a user who has never used the platform before — it tells them which button to press next. A badge that lingers on the wrong engine after a stage completes actively misleads the user's workflow and undermines the UX investment made in this changeset.

3. [LOGIC-003] / [HARP-AI-002] — Engine id/name misalignment and magic `id <= 7` boundary are a compounding future-defect factory

- **Severity:** HIGH (3/5 each; combined blast radius is HIGH)
- **Blast Radius:** HIGH — The `id <= 7` guard in `nextEngineId` and the F8 `name / id: 9` mismatch are not isolated to one component. They are load-bearing assumptions encoded implicitly in `session-detail.tsx`, relied upon by `FeatureNavItem`'s `isNext` prop contract, and will affect any future developer or AI agent that adds, reorders, or queries engines. The `data-testid="feature-nav-f8"` selector resolves from `name.toLowerCase() = "f8"` on the entry with `id: 9` — so test infra already depends on the name-over-id convention without documenting it.
- **Estimated Fix Time:** 45–90 minutes (Refactor)
- **Fix Pathway:** Refactor — add `isPipelineStage: boolean` (or `isSideStep: boolean`) to each `ENGINES` entry; replace `e.id <= 7` with `e.isPipelineStage` (or `!e.isSideStep`) throughout; add a JSDoc block above `ENGINES` stating the semantic contract; optionally renumber F8's `id` from 9 to 8 to align name and id, updating F6-VDJ's id to a non-colliding value (e